

PLADICS: A Lightweight YOLO26n-Based Framework for River-Surface Plastic Waste Detection and Collection

Mst. Esat Jahan Akhi¹, Jannatul Ferdushi Ayesha², Suman Saha^{3*}

^{1,2,3*}Department of Internet of Things and Robotics Engineering,
University of Frontier Technology, Bangladesh, Gazipur-1750,
Bangladesh.

*Corresponding author(s). E-mail(s): suman0001@uftb.ac.bd;
Contributing authors: esat0001@std.uftb.ac.bd;
Jannatul0001@std.uftb.ac.bd;

Abstract

River-borne plastic waste continues to damage freshwater ecosystems, creating a clear need for monitoring approaches that can operate reliably at scale. However, many existing vision-based studies rely largely on dataset-specific evaluations and provide limited evidence regarding annotation quality, robustness, confidence reliability, and model interpretability. This study presents PLADICS, an integrated framework for riverine plastic detection with collection-oriented system integration. A detailed dataset audit reduced 7,759 image-label pairs to 7,264 and removed 175 invalid bounding boxes. The cleaned single-class dataset was then used to compare YOLOv8n, YOLOv10n, YOLO11n, YOLO26n, YOLO26s, NanoDet-Plus-M, and RT-DETR-L, while separate multiclass experiments examined category granularity and transfer learning. The evaluation also included four-fold internal cross-validation, reduced-training-data analysis, confidence calibration, and detection-specific SHAP attribution. On the held-out test set, the 100-epoch YOLO26n checkpoint produced 0.919 Precision, 0.844 Recall, 0.915 mAP@0.5, and 0.608 mAP@0.5:0.95. Four-fold validation resulted in a mean mAP@0.5 of **0.860 ± 0.015**, while the reduced-data experiments showed a progressive decline in performance as the amount of training data decreased. The threshold-conditioned ECE was 0.1696, and the SHAP analysis showed strong deletion sensitivity but only partial spatial concentration within the detected regions. The selected detector was then connected to ByteTrack-assisted target

continuity and rule-based collection logic. Overall, PLADICS provides a reproducible framework for evaluating and integrating river-surface plastic detection without assuming that the results will generalize universally to unseen waterways.

Keywords: River-surface plastic detection, Riverine plastic, Lightweight object detection, YOLO26n, ByteTrack, Robotic collection

1 Introduction

Plastic pollution in rivers and inland-water systems has become a persistent environmental concern because of its effects on aquatic ecosystems, biodiversity, water quality, and downstream marine environments [1, 2]. Global plastic production now exceeds 430 million tonnes annually, and weaknesses in waste collection, disposal, and recycling allow part of this material to escape into land and water systems [2]. Rivers play an important role in transporting mismanaged plastic from inland and urban areas toward lakes, estuaries, and oceans. An estimated 1,000-plus rivers are responsible for almost 80% of river-derived plastic entering the oceans, making upstream monitoring and interception particularly relevant [3]. Once present in rivers, plastic debris can obstruct waterways, interact with aquatic organisms through ingestion or entanglement, and gradually fragment into smaller particles that persist in freshwater environments [1]. Continuous monitoring and removal, however, remain difficult because manual inspection and collection require considerable labor, time, and operational resources [4]. Natural river environments also introduce visual challenges such as reflections, glare, changing illumination, vegetation, water motion, occlusion, and background clutter, making reliable automated plastic detection difficult.

Researchers have explored a range of sensing and computational methods for aquatic-waste monitoring. Camera-based observation has increasingly been paired with deeplearning models that learn object representations directly from annotated visual data. Convolutional neural network (CNN) detectors and one-stage object detectors have been widely used for real-time visual recognition, while lightweight YOLO-based architectures are particularly attractive when detection accuracy must be balanced against computational cost [5, 6]. Recent YOLO-based methods have improved small-object and water-surface detection through lightweight backbones, multi-scale feature fusion, attention mechanisms, and architecture optimization [7, 8]. Transformer-based detectors such as RT-DETR provide another direction by using global contextual representation and end-to-end detection [9]. For video-based systems, tracking algorithms can maintain target continuity across frames and support decisions based on persistent object identities rather than isolated detections. Edge-computing platforms enable these models to operate closer to the sensing device, while IoT communication can support telemetry, remote monitoring, and mission logging [10]. When these technologies are connected with robotic navigation and mechanical collection mechanisms, visual detection can form part of a complete autonomous or semi-autonomous waste-management workflow rather than serving only as an image-analysis task.

Despite recent progress, several practical gaps remain. Most studies evaluate detector accuracy within a particular dataset or model family, which makes direct comparison difficult when datasets, class definitions, hardware, and evaluation settings differ. Dataset quality is another concern because annotation errors and incorrect class mappings can affect both training and evaluation. Robustness across different data partitions or reduced training sets is also less frequently examined. In addition, confidence calibration and model explainability receive limited attention, even though these factors become important when detection results are used to guide physical actions. A complete riverine-plastic management system must also connect perception with target tracking, embedded inference, navigation, collection, telemetry, and monitoring. These aspects are rarely evaluated together within the same experimental framework.

To address these gaps, we developed PLADICS (Plastic Detection and Intelligent Collection System) as an integrated framework for river-surface plastic detection and collection. The system combined dataset refinement, detector comparison, single- and multiclass evaluation, robustness analysis, confidence calibration, SHAP-based interpretation, embedded inference, target continuity, rover navigation, physical collection, and IoT monitoring. Our aim was to evaluate the detection model beyond conventional accuracy measures and then examine how the selected lightweight model could be used within a resource-constrained physical prototype. The main contributions of this work are:

- We developed PLADICS, an integrated river-surface plastic detection and collection framework that combined deep learning-based perception, embedded inference, robotic control, physical collection, and IoT monitoring within a unified system.
- We rebuilt the riverine-plastic datasets after correcting annotation and class-mapping errors, generated single-class and multiclass variants, and compared YOLO, compact CNN, and transformer detectors to select an efficient lightweight candidate.
- We evaluated the detection framework beyond conventional benchmark metrics through transfer-learning analysis, cross-validation, reduced-data robustness experiments, confidence calibration, and SHAP-based attribution analysis to examine performance consistency, reliability, and model behavior.
- We deployed the selected YOLO26n model on a Raspberry Pi using NCNN and integrated ByteTrack-assisted target continuity, rule-based rover navigation, collection actuation, telemetry, and mission logging to demonstrate an end-to-end detection-to-collection prototype workflow.

Section 2 summarizes the relevant literature, followed by the PLADICS methodology in Section 3. Experimental findings and prototype evaluation are reported in Section 4, and Section 5 closes the paper with limitations and directions for further work.

2 Literature Review

Recent aquatic-waste research has drawn on lightweight detection networks, transformer architectures, video tracking, and embedded vision. YOLO models are common

in this setting because they combine relatively low computational demand with competitive detection performance, whereas compact CNNs and transformer detectors occupy different points on the accuracy–complexity spectrum [6, 11]. Public benchmarks such as FloW and RiSID have further supported progress in this field by providing annotated river-surface and inland-water imagery for floating-waste detection under different category configurations [12–14].

A major line of work has focused on adapting lightweight YOLO detectors to difficult water-surface scenes. Chen and Zhu [7] combined ShuffleNetV2, GSConv, and attention within YOLOv5 to reduce computational load. Fan et al. [8] targeted multiclass surface litter with SS-YOLOv8n, whereas Wang et al. [15] strengthened multi-scale representation in YOLOv8. Peng et al. [16] instead modified YOLOv7 with additional fusion and attention components for improved localization of floating debris.

Table 1: Representative vision-based approaches for aquatic and riverine waste detection.

Study	Model	Dataset	mAP	Key Limitation
Chen and Zhu (2024) [7]	Improved Lightweight YOLOv5	Orca Flow-Img Dataset	0.849	Limited robustness under strong illumination and reflective water conditions.
Fan et al. (2024) [8]	SS-YOLOv8n	SLD dataset	0.799	Performance remained affected by reflections, semi-floating objects, and complex scenes.
Wang et al. (2024) [15]	YOLOv8-MSS	FloW-Img	0.879	Water-wave interference and object-scale variation remained challenging.
Peng et al. (2024) [16]	Improved YOLOv7	River floating-waste dataset	0.863	Additional modules increased computational complexity.
Nunkhaw et al. (2024) [17]	YOLOv5 + DeepSORT	Laboratory flume dataset	0.880	Evaluation was limited to controlled laboratory conditions.
Tikász et al. (2025) [18]	YOLOv8s	River and UAV macroplastic dataset	0.770	Small and distant objects remained difficult under occlusion.
Sun et al. (2025) [19]	RTDETR-MARD	FloW and custom datasets	0.866	Higher computational demand limited embedded deployment suitability.
Badams et al. (2025) [20]	FloYO-Net	D.six dataset	0.828	Limited dataset diversity restricted generalization analysis.
Cao et al. (2026) [21]	SEDS-YOLOv8	Hybrid water-surface dataset	0.888	Generalization beyond augmented training conditions requires further evaluation.
Sumon et al. (2026) [22]	RT-DETR-X + Faster R-CNN Ensemble	FloW-Img	0.891	Ensemble inference increases computational requirements.
Xu et al. (2026) [23]	YOLOv12n	Nine-category riverine debris dataset	0.712	Multi-category recognition increases classification difficulty.
Jitnupong et al. (2026) [24]	YOLOv11	Floating-waste dataset	0.780	Small dataset size limited broader validation.
Zhou et al. (2026) [25]	ESAK-YOLO	Complex floating-waste dataset	0.889	Occlusion and background interference remained challenging.

Note: Values are reproduced from the cited studies and should not be interpreted as controlled head-to-head comparisons because the datasets, class definitions, splits, hardware, and evaluation protocols differ across studies.

Transformer-based and hybrid approaches have also been investigated to improve contextual understanding and multi-scale representation. Zhang et al. [26] developed LR-DETR for river floating-object detection, while Sun et al. [19] proposed RTDETR-MARD using transformer-based detection with multi-scale feature processing. Beyond frame-level detection, tracking-assisted methods have attempted to maintain object

continuity across consecutive frames. Nunkhaw et al. [17] combined YOLOv5 with DeepSORT for floating-waste tracking, whereas Tikász et al. [18] evaluated YOLOv8s using riverbank and UAV-based macroplastic imagery. More recent studies have explored larger datasets, ensemble strategies, and improved YOLO variants. Badams et al. [20] introduced FloYO-Net for small floating-waste detection, Cao et al. [21] proposed SEDS-YOLOv8 using a large augmented water-surface dataset, and Sumon et al. [22] combined RT-DETR-X and Faster R-CNN predictions through weighted box fusion. Xu et al. [23] investigated YOLOv12n for multi-category riverine debris detection, Jitnupong et al. [24] applied YOLOv11 for floating-waste detection, classification, and counting, and Zhou et al. [25] introduced ESAK-YOLO using multi-scale and attention-based feature enhancement for complex water environments.

Across these studies, several common challenges remain, including small-object detection, water reflection, occlusion, dynamic backgrounds, and differences between evaluation protocols. Although recent methods have improved detection accuracy, most studies primarily focus on detector-level performance under specific datasets and experimental settings. A unified evaluation combining dataset refinement, detector-family comparison, reduced-data robustness, confidence calibration, explainability, and deployment-oriented analysis remains limited. In addition, the transition from visual detection to autonomous target selection, navigation, physical collection, and mission-level monitoring has received comparatively less attention.

PLADICS is designed to address these limitations by combining dataset refinement with comparative evaluation of lightweight YOLO, compact CNN, and transformer-based detectors under a common framework. The study further investigates model robustness, confidence reliability, attribution behavior, and computational efficiency. The selected lightweight detector is integrated with ByteTrack-based target continuity, Raspberry-Pi-based rover control, plastic collection, telemetry, and mission logging to demonstrate an end-to-end river-surface plastic detection and collection prototype.

3 Methodology

The PLADICS methodology covers dataset preparation, detector evaluation, onboard deployment, rover control, plastic collection, and IoT monitoring. The complete workflow is shown in Fig. 1.

3.1 Dataset Description

The experiments drew on two public sources: RiSID and the Riverine Plastic Litter Dataset. RiSID provides 7,356 images and 8,022 labeled debris instances captured at 11 locations spanning seven Japanese rivers, with COCO annotations available at multiple category granularities [13]. The Riverine dataset labels cans, plastic bags, bottles, boxes, and cups [27]. RiSID formed the main data source, and Riverine images were added to increase the plastic examples available for the merged single-class set.

3.1.1 Dataset Cleaning and Annotation Refinement

Before training, custom Python scripts were used to audit image-label correspondence, missing or empty labels, image readability, class IDs, annotation structure,

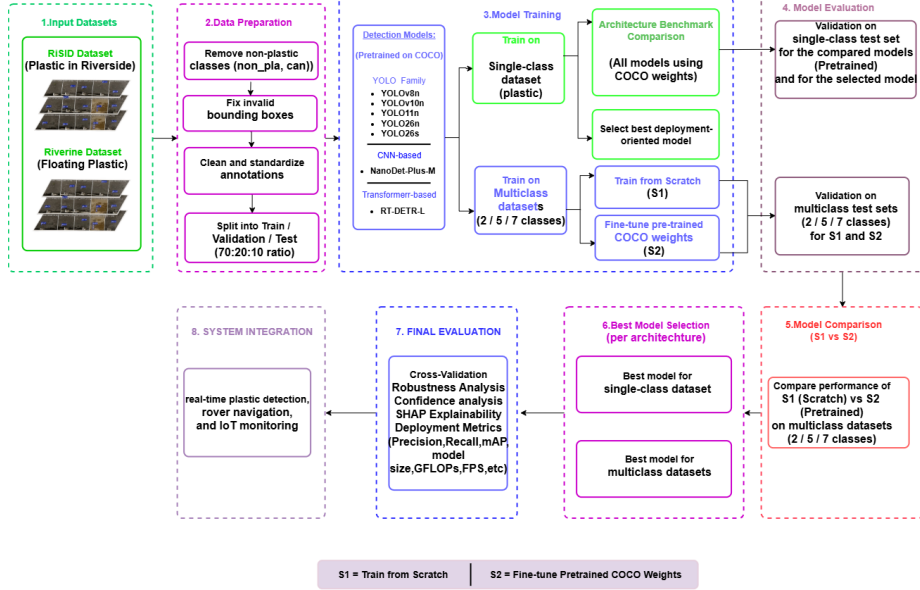


Fig. 1: Overall methodology framework of the proposed PLADICS system, showing dataset preparation, single-class and multiclass dataset construction, model training, architecture comparison, evaluation, deployment model selection, and rover-based system integration.

and bounding-box validity. Detected annotation and class-mapping errors were corrected before dataset reconstruction [28]. Among the available Riverine annotations, 142 XML files had corresponding images in the `photo` and `test` directories. Only the plastic categories were retained, while `can` was excluded. For RiSID, only `pla` annotations were retained because the initial single-class conversion had also mapped `non_pla` to plastic. The cleaned datasets were regenerated from the original COCO and XML annotations. Out-of-bound boxes were clipped when a valid positive-area region remained; otherwise, they were removed. Source-specific filename prefixes prevented collisions during merging.

The resulting audit summary is reported in Table 2.

3.1.2 Dataset Construction

Two dataset types were prepared. For single-class detection, all retained RiSID and Riverine plastic annotations were mapped to class 0, `plastic`. The RiSID two-, five-, and seven-class datasets were generated separately using the original category definitions [13]. Single-class detection was used for rover deployment because control depends on target presence, horizontal position, and bounding-box area rather than plastic subtype. The multiclass configurations are listed in Table 3.

Table 2: Dataset audit summary before and after refinement.

Metric	Before	After
Images	7759	7264
Labels	7759	7264
Missing labels	0	0
Empty labels	0	0
Invalid boxes	175	0
Corrupt images	0	0
Invalid class IDs	0	0

Table 3: RiSID multiclass configurations used in the experiments.

Configuration	Categories
RiSID 2-class	<code>pla, non_pla</code>
RiSID 5-class	<code>bottle, food_pack, bag, o_pla, non_pla</code>
RiSID 7-class	<code>bottle, o_bottle, food_pack, bag, o_bag, o_pla, non_pla</code>

3.2 Detection Models

The single-class benchmark included YOLOv8n, YOLOv10n, YOLO11n, YOLO26n, and YOLO26s. NanoDet-Plus-M and RT-DETR-L were added as compact CNN and transformer-based baselines. YOLO and RT-DETR experiments were run through Ultralytics, whereas NanoDet-Plus-M was trained and evaluated with its original codebase [9, 29–32].

3.2.1 YOLO26 Architecture

YOLO26 was included as a lightweight detector candidate in this study. The pre-trained YOLO26n and YOLO26s checkpoints were evaluated without modification, and the selected YOLO26n checkpoint was exported to NCNN format for embedded deployment [31].

3.3 Onboard Detection, Rover Navigation, and Plastic Collection

The selected YOLO26n-100 model was exported to NCNN and deployed on a Raspberry Pi 4 with a Camera Module V2. Frames were captured at 640×480 pixels and processed at an inference size of 416 pixels. ByteTrack with persistent tracking was used to associate detections across frames.

Only detections belonging to the target class `plastic` were considered for navigation. A candidate i was accepted when

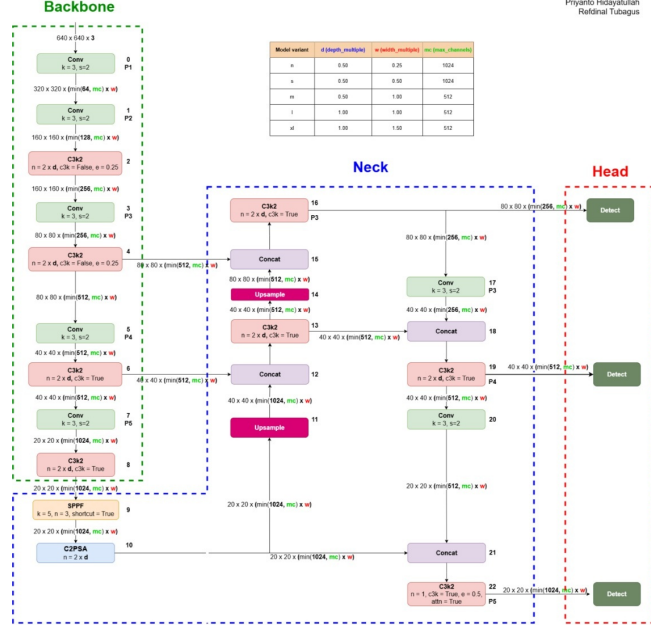


Fig. 2: Schematic overview of the YOLO26 detection architecture used in this study, based on the official model description [31].

$$C_i \geq \tau_{\text{acc}}, \quad A_i \geq A_{\text{min}}, \quad \frac{A_i}{WH} \leq \rho_{\text{max}}, \quad (1)$$

Here, C_i denotes confidence and A_i denotes the candidate box area. The frame dimensions are W and H ; τ_{acc} sets the minimum confidence, A_{min} sets the minimum box area, and ρ_{max} prevents excessively large boxes from being accepted.

For each accepted candidate, the horizontal center and bounding-box area were computed as

$$c_{x,i} = \frac{x_{1,i} + x_{2,i}}{2}, \quad A_i = (x_{2,i} - x_{1,i})(y_{2,i} - y_{1,i}). \quad (2)$$

When multiple candidates were available, the primary target was selected using

$$S_i = w_C C_i + w_A \tilde{A}_i + w_G G_i + w_T T_i, \quad w_C + w_A + w_G + w_T = 1, \quad (3)$$

where

$$\tilde{A}_i = \frac{A_i}{\max_j A_j}, \quad G_i = \max \left(0, 1 - \frac{|c_{x,i} - W/2|}{W/2} \right). \quad (4)$$

The temporal-continuity term was defined as

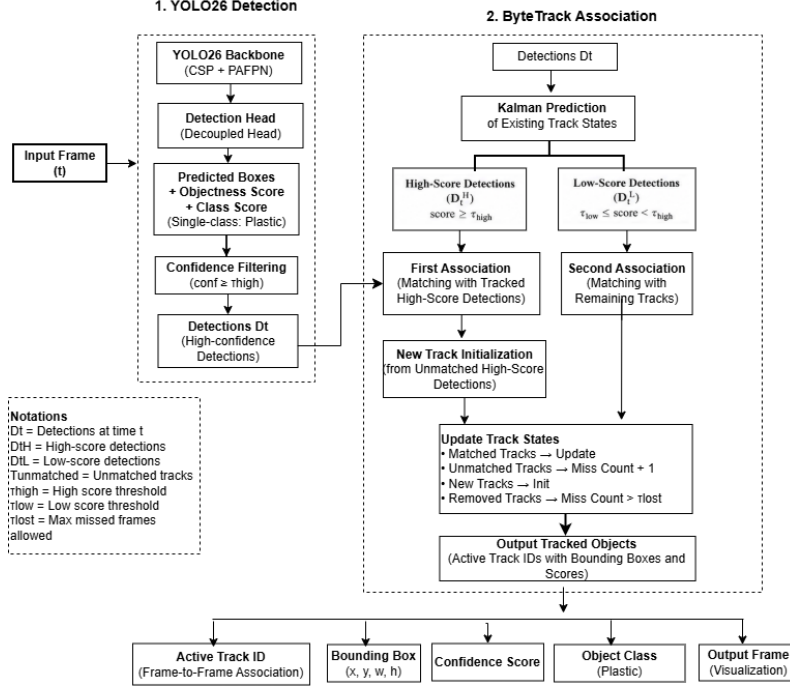


Fig. 3: YOLO26n-ByteTrack onboard detection pipeline.

$$T_i = \begin{cases} \eta_{id}, & \text{if the valid Track ID is unchanged,} \\ \eta_{prox}, & d_i \leq d_{max} \wedge r_{min} \leq A_i/A_{t-1} \leq r_{max}, \\ 0, & \text{otherwise,} \end{cases} \quad (5)$$

where d_i is the displacement from the previously selected target. The second condition provides spatial continuity when a valid Track ID is unavailable; it does not recover or reassign a ByteTrack identity.

If no current candidate was available, the last valid target state was retained for at most N_{hold} processed frames. This retained state was used only for short-term navigation continuity and was not permitted to trigger collection.

The rover navigation command was determined by the horizontal target position and projected bounding-box area:

$$a_t = \begin{cases} \text{STOP,} & A_t \geq A_{stop}, \\ \text{TURN_LEFT,} & A_t < A_{stop} \wedge c_{x,t} < \lambda_L W, \\ \text{TURN_RIGHT,} & A_t < A_{stop} \wedge c_{x,t} > \lambda_R W, \\ \text{MOVE_FORWARD,} & \text{otherwise.} \end{cases} \quad (6)$$

If no target remained after the retention period, the rover entered the **SEARCH** state. Movement commands were smoothed over the recent action history, while **STOP** was

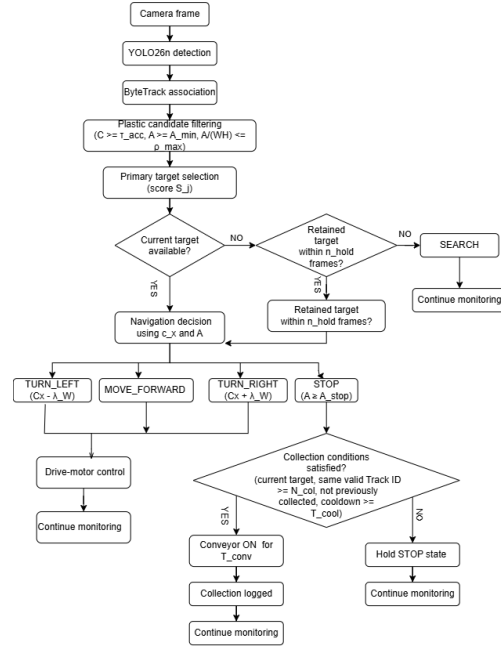


Fig. 4: Rule-based target selection, navigation, and collection control in the PLADICS rover.

applied immediately. The L298N controller then translated the selected command into forward or turning motion; during **SEARCH**, both motors remained stopped.

Collection was triggered only when the rover was in the **STOP** state and the current target retained the same valid Track ID for at least N_{col} consecutive frames, had not been previously collected, and satisfied the cooldown condition. The conveyor was then activated for T_{conv} . Deployment parameters are listed in Table 4.

Table 4: Parameters used in the onboard PLADICS deployment.

Parameter	Description	Value
τ_{det}	Detector confidence threshold	0.20
τ_{IoU}	Detector IoU threshold	0.45
τ_{acc}	Candidate acceptance confidence	0.40
A_{min}	Minimum accepted bounding-box area	1200 px
ρ_{max}	Maximum box-to-frame area ratio	0.55
w_C, w_A, w_G, w_T	Target-selection weights	0.40, 0.30, 0.15, 0.15
η_{id}	Same-ID continuity score	1.00
η_{prox}	Spatial-continuity score	0.65
d_{max}	Maximum continuity displacement	100 px
$r_{\text{min}}, r_{\text{max}}$	Accepted area-ratio range	0.45, 2.20
N_{hold}	Target-retention window	6 frames
λ_L, λ_R	Navigation-zone boundaries	0.33, 0.66
A_{stop}	Stop-area threshold	45000 px
N_{smooth}	Action-smoothing window	5 frames
N_{col}	Collection stability requirement	4 frames
T_{cool}	Collection-event cooldown	5 s
T_{conv}	Conveyor activation duration	5 s

3.4 IoT Monitoring and Data Logging

PLADICS used a local Mosquitto broker and MQTT messaging to transmit system telemetry to a Node-RED dashboard [33, 34]. Each mission generated a timestamped CSV log, while the annotated video stream was provided through a Flask MJPEG endpoint. The main monitored parameters are summarized in Table 5.

Table 5: Main parameters monitored and logged by PLADICS.

Parameter	Description
Detection count	Number of accepted plastic detections
Track ID	Active ByteTrack identifier
Confidence score	Confidence of the selected target
Navigation action	Current rover command
Mission state	Current operational state
Conveyor state	ON/OFF state of the collection mechanism
Inference time	Per-frame processing time
Frame rate	Processed frames per second



Fig. 5: figure

Node-RED dashboard interface for real-time PLADICS monitoring.

4 Experimental Results

4.1 Experimental Setup

YOLO and RT-DETR-L were executed with Ultralytics and PyTorch, whereas NanoDet-Plus-M retained its original evaluation pipeline. All benchmark runs were CPU-based on Windows 11 using an Intel[®] Core[™] i5-1235U system with 16 GB RAM. The software stack comprised Python 3.11.9, PyTorch 2.10.0+cpu, Torchvision 0.25.0, OpenCV 4.13.0, and Ultralytics 8.4.36.

Except where an experiment required a different setting, training used 640×640 inputs, batch size 8, and 50 epochs. YOLO26n was additionally extended to 100 epochs. Clean datasets followed a 70:20:10 train-validation-test division. Test subsets were used for the principal benchmark, multiclass, transfer-learning, robustness, calibration, and final-model evaluations, while cross-validation used only the held-out validation fold and did not reuse the original test set.

Detector quality was measured with Precision, Recall, mAP@0.5, and mAP@0.5:0.95. Parameter count, GFLOPs, checkpoint size, inference latency, and FPS were recorded as efficiency indicators. These timing measurements refer to the CPU workstation rather than the Raspberry Pi deployment platform. Table 6 lists the main benchmark settings.

Table 6: Default configuration used for the Ultralytics-based benchmark experiments.

Parameter	Value
Framework	Ultralytics 8.4.36 (YOLO and RT-DETR-L)
Python Version	3.11.9
Operating System	Windows 11 (64-bit)
Deep Learning Library	PyTorch 2.10.0+cpu
Annotation Format	YOLO detection format
Input Image Size	640 × 640
Batch Size	8
Training Epochs	Experiment-dependent: 20, 50, or 100
Dataset Split	70% training, 20% validation, 10% test
Device	CPU

4.2 Performance on Unrefined and Refined Datasets

Dataset refinement was assessed by training the same detector set separately on the original merged data and on the reconstructed clean version. Performance was then measured on the held-out split associated with each dataset state.

Table 7: Test-set performance before and after dataset refinement.

Dataset State	Metric	YOLO10n	YOLO8n	YOLO11n	YOLO26n	YOLO26s
Before Cleaning	Precision	0.635	0.734	0.851	0.836	0.844
	Recall	0.688	0.692	0.745	0.738	0.722
	mAP@0.5	0.671	0.728	0.807	0.793	0.786
	mAP@0.5:0.95	0.339	0.375	0.424	0.461	0.474
After Cleaning	Precision	0.623	0.858	0.668	0.859	0.859
	Recall	0.718	0.797	0.746	0.797	0.813
	mAP@0.5	0.693	0.868	0.767	0.866	0.876
	mAP@0.5:0.95	0.431	0.562	0.482	0.556	0.580

Note: All reported values are test-set results. The before-cleaning results were obtained from the unrefined dataset test split, whereas the after-cleaning results were obtained from the reconstructed clean dataset test split. Since dataset refinement removed contaminated and invalid samples, the two test subsets differ in size and composition.

The cleaned-data runs in Table 7 generally produced higher detection scores. YOLOv8n obtained 0.868 mAP@0.5 and 0.562 mAP@0.5:0.95, YOLO26n reached 0.866 and 0.556, and YOLO26s recorded the highest 50-epoch cleaned-data values at 0.876 and 0.580, respectively. However, because the refinement process changed the

number and composition of test samples, the differences represent the combined outcome of annotation correction, class filtering, and dataset reconstruction rather than the isolated effect of a single cleaning operation.

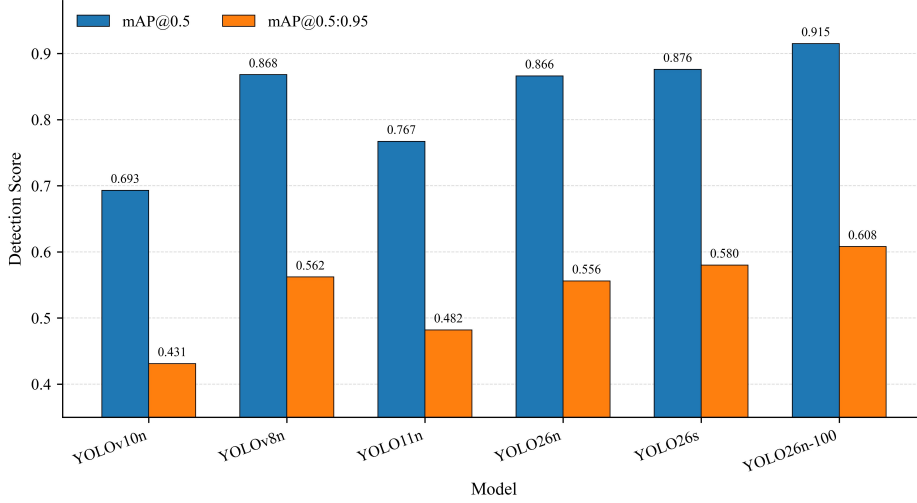


Fig. 6: Comparison of lightweight YOLO model performance after dataset refinement under the 50-epoch benchmark setting.

Fig. 6 summarizes the lightweight YOLO model performance under the 50-epoch refined-dataset benchmark. YOLOv8n, YOLO26n, and YOLO26s achieved the strongest performance among the evaluated lightweight models.

4.3 Architecture Comparison Across Detection Model Families

The cleaned single-class dataset was also used for a cross-family benchmark covering YOLO detectors, the compact CNN-based NanoDet-Plus-M, and the transformer-based RT-DETR-L. Both detection quality and computational cost were considered through accuracy metrics, parameters, GFLOPs, checkpoint size, latency, and FPS.

A 20-epoch training budget was used for this comparison. Because RT-DETR-L and NanoDet-Plus-M followed their framework-specific implementations, the experiment should be viewed as a practical cross-architecture benchmark rather than a fully controlled implementation-level ablation.

Table 8 presents the 20-epoch architecture-level results. RT-DETR-L achieved the highest detection accuracy among the models with complete Precision and Recall reporting, reaching 0.858 mAP@0.5 and 0.554 mAP@0.5:0.95. It also obtained a Precision of 0.861 and a Recall of 0.808.

Within the YOLO group, YOLO26s reached 0.655 mAP@0.5 and 0.384 mAP@0.5:0.95. YOLO26n was only slightly lower at 0.650 and 0.382, but used far

Table 8: Twenty-epoch architecture-level comparison on the cleaned merged single-class dataset

Model	Architecture	Params	GFLOPs	P	R	mAP50	mAP50-95	FPS	Infer. (ms)	Size (MB)
RT-DETR-L	HGNetv2-L + Transformer Decoder	31.99M	103.4	0.861	0.808	0.858	0.554	0.98	1021.41	63.14
NanoDet-Plus-M	ShuffleNetV2 + GhostPAN	1.17M	3.69	–	–	0.615	0.354	1.51	664.00	16.27
YOLO11n	YOLO	2.58M	6.30	0.563	0.628	0.615	0.346	6.81	146.77	5.21
YOLOv8n	YOLO	3.01M	8.10	0.618	0.661	0.648	0.371	6.33	158.05	5.95
YOLO26n	YOLO	2.38M	5.20	0.616	0.637	0.650	0.382	5.26	190.20	5.12
YOLO26s	YOLO	9.47M	20.50	0.642	0.618	0.655	0.384	2.20	454.23	19.36

Note: Inference time is reported in milliseconds per image, and model size is reported in MB. NanoDet-Plus-M reports COCO-style AP metrics, while Precision and Recall were not directly provided by its native evaluation pipeline. The reported model sizes correspond to native framework checkpoint files and may include different serialization overheads.

fewer parameters and substantially less computation than YOLO26s. Under this 20-epoch setting, the YOLO26 variants also produced slightly higher mAP values than YOLOv8n and YOLO11n.

NanoDet-Plus-M had the lowest parameter count and GFLOPs, but its reported AP values remained below those of the YOLO26 variants. RT-DETR-L achieved the strongest accuracy but required 31.99M parameters, 103.4 GFLOPs, a 63.14 MB checkpoint, and the highest CPU inference latency. The comparison therefore shows that higher accuracy was generally accompanied by greater computational demand across the tested architectures.

Considering detection performance, model complexity, checkpoint size, and CPU latency, YOLO26n provided a favorable balance for the resource-constrained objectives of PLADICS. It was therefore selected for further refinement.

4.4 Multiclass Riverine Plastic Detection

Although the main deployment model follows a single-class plastic detection strategy, multiclass experiments were also conducted to study category-level behavior. For this purpose, YOLO26n was evaluated on cleaned RiSID 2-class, 5-class, and 7-class datasets. The objective was to examine class-level performance under different category configurations.

Table 9 shows that class-wise performance varied across the different category definitions. In the RiSID 7-class setting, Bottle achieved 0.881 mAP@0.5 and Bag achieved 0.860 mAP@0.5, whereas O_bottle and O_bag produced lower scores of 0.518 and 0.660, respectively. The results show variation among categories, indicating that some classes remained more challenging under the evaluated conditions.

Fig. 7 shows that overall mAP@0.5 decreased from 0.815 in the 2-class configuration to 0.781 and 0.755 in the 5-class and 7-class configurations, respectively. In contrast, mAP@0.5:0.95 remained comparable across the three configurations, indicating different effects of category granularity across the two mAP criteria.

Table 9: Class-wise multiclass detection performance using YOLO26n.

Dataset	Class	P	R	mAP50	mAP50-95
RiSID-2	Plastic	0.820	0.860	0.893	0.587
	Non-plastic	0.783	0.654	0.736	0.457
	Overall	0.801	0.757	0.815	0.522
RiSID-5	Bottle	0.724	0.727	0.781	0.527
	Food_pack	0.684	0.606	0.753	0.583
	Bag	0.599	0.636	0.652	0.420
	O_pla	0.780	0.804	0.831	0.539
	Non_pla	0.776	0.720	0.774	0.471
	Overall	0.724	0.727	0.781	0.527
RiSID-7	Bottle	0.788	0.819	0.881	0.645
	O_bottle	0.663	0.435	0.518	0.375
	Food_pack	0.708	0.667	0.758	0.609
	Bag	0.683	0.851	0.860	0.636
	O_bag	0.628	0.604	0.660	0.457
	O_pla	0.768	0.788	0.840	0.532
	Non_pla	0.817	0.717	0.768	0.521
	Overall	0.722	0.697	0.755	0.539

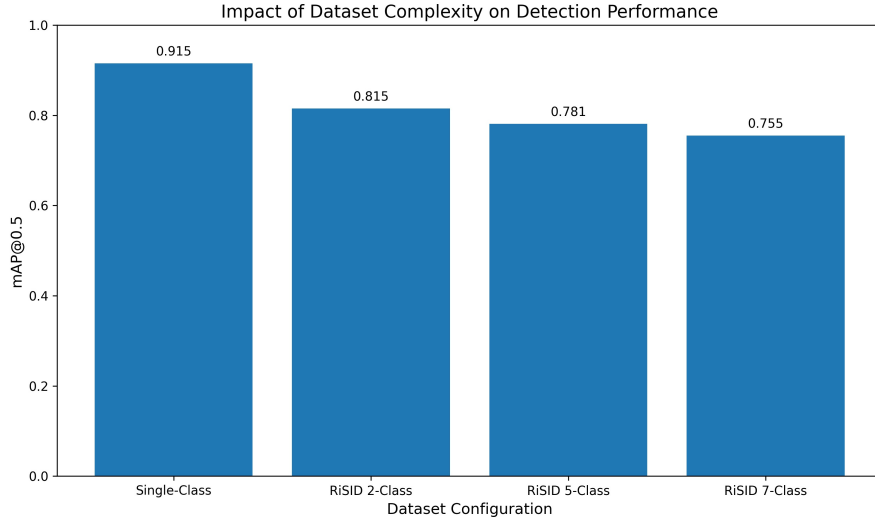


Fig. 7: Configuration-level detection performance for the single-class and RiSID multiclass experiments.

4.5 Transfer Learning Analysis

The transfer-learning experiment compared YOLO26n trained from scratch (S1) with the same model initialized from pretrained COCO weights (S2) on the RiSID 5-class configuration.

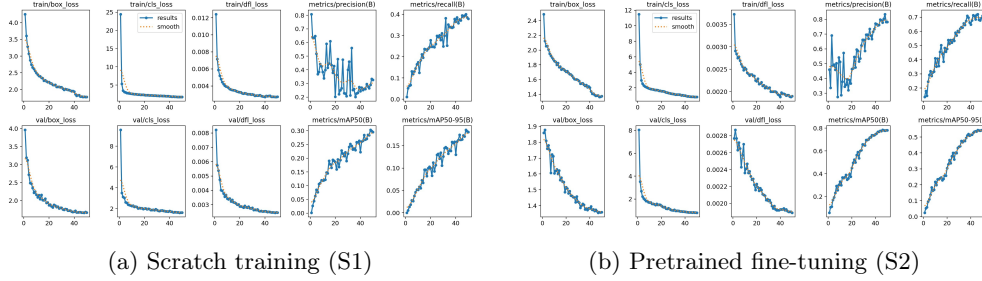


Fig. 8: Training convergence comparison between scratch training and pretrained fine-tuning strategies.

Table 10: Scratch training versus pretrained fine-tuning performance.

Training Strategy	P	R	mAP50	mAP50-95
S1 (Scratch Training)	0.290	0.401	0.303	0.185
S2 (Pretrained Fine-Tuning)	0.724	0.727	0.781	0.527

Pretrained initialization produced a large gain over scratch training: mAP@0.5 rose from 0.303 to 0.781, while mAP@0.5:0.95 increased from 0.185 to 0.527 (Table 10). The training curves in Fig. 8 also show slower convergence and weaker validation performance for scratch training, while the pretrained model reached higher and more stable validation metrics.

4.6 Cross-Validation Analysis

The stability of the 50-epoch YOLO26n setting was further checked across four independently held-out validation folds.

Across the four folds, the mean Precision, Recall, mAP@0.5, and mAP@0.5:0.95 were 0.8311, 0.8028, 0.8596, and 0.5522. Fold-level mAP@0.5 varied between 0.8345 and 0.8750, with only a small standard deviation (Table 11). The results indicate consistent performance across the evaluated internal partitions, although they do not establish generalization to unseen locations or sensing conditions.

Table 11: Four-fold cross-validation performance of the YOLO26n configuration.

Fold	P	R	mAP@0.5	mAP@0.5:0.95
Fold-1	0.8660	0.8200	0.8750	0.5711
Fold-2	0.8300	0.7907	0.8631	0.5497
Fold-3	0.7913	0.7746	0.8345	0.5426
Fold-4	0.8371	0.8257	0.8658	0.5456
Mean $\pm$ Std	0.8311 $\pm$ 0.0267	0.8028 $\pm$ 0.0210	0.8596 $\pm$ 0.0152	0.5522 $\pm$ 0.0112

Note: All folds were trained for 50 epochs. Metrics were calculated on the held-out validation partition of each fold. The reported values represent the mean and population standard deviation across four completed folds, while the original test subset was excluded from cross-validation.

4.7 Robustness Under Reduced-Data Conditions

To test sensitivity to training-set size, YOLO26n was retrained with 75%, 50%, and 25% of the available training images; the validation and test data were kept fixed.

Table 12: Reduced-data robustness of the 50-epoch YOLO26n configuration.

Training Data	P	R	mAP@0.5	mAP@0.5:0.95
100%	0.859	0.797	0.866	0.556
75%	0.818	0.777	0.844	0.532
50%	0.690	0.707	0.742	0.460
25%	0.567	0.667	0.632	0.379

Table 12 and Fig. 9 show a progressive decline in performance as the training set was reduced. mAP@0.5 decreased only slightly from 0.866 to 0.844 at the 75% level, but fell to 0.742 and 0.632 with 50% and 25% of the training data, respectively. The results indicate that moderate data reduction had a limited effect, whereas larger reductions produced a clear performance loss.

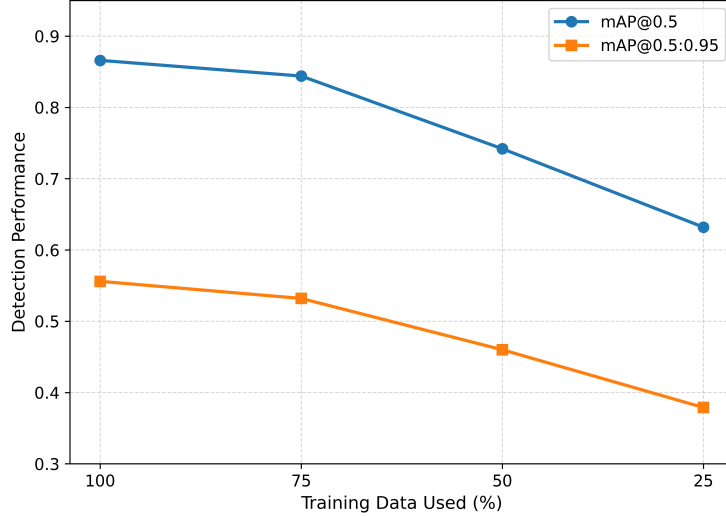


Fig. 9: Detection performance trend under reduced-data conditions.

4.8 Final Lightweight Model Refinement

YOLO26n was chosen for longer training after the preceding benchmarks showed competitive accuracy at relatively low model complexity. Its training schedule was therefore extended from 50 to 100 epochs.

Table 13: Performance of YOLO26n under 50- and 100-epoch training configurations.

Model	P	R	mAP50	mAP50-95
YOLO26n (50 epochs)	0.859	0.797	0.866	0.556
YOLO26n (100 epochs)	0.919	0.844	0.915	0.608

Extending training to 100 epochs improved all four reported metrics (Table 13). Precision changed from 0.859 to 0.919, Recall from 0.797 to 0.844, mAP@0.5 from 0.866 to 0.915, and mAP@0.5:0.95 from 0.556 to 0.608. Under the current training configuration, the 100-epoch run produced higher test-set scores than the 50-epoch run, indicating that the longer training schedule was beneficial in this experiment. The training and validation curves did not show severe late-epoch divergence.

The 100-epoch curves in Fig. 10 show continued improvement in the validation metrics while the training and validation trajectories remain broadly aligned. The resulting checkpoint was therefore used in the later analyses.

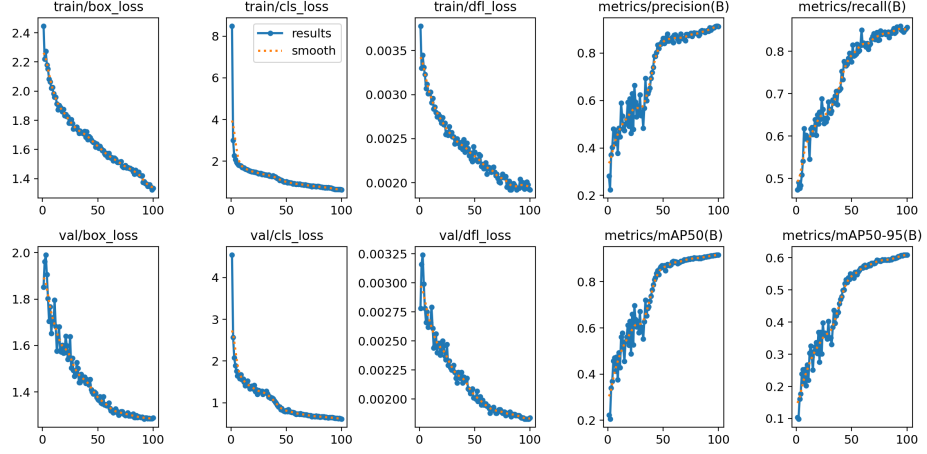


Fig. 10: Training and validation behavior of the refined YOLO26n model during 100-epoch optimization.

The confusion matrix in Fig. 11 provides an error-oriented view of the YOLO26n-100 predictions. Most plastic instances were correctly detected, while the remaining errors were associated with missed detections and background-related false responses.

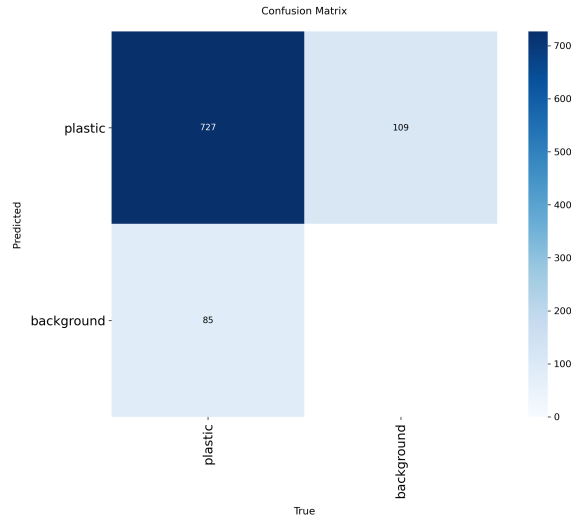


Fig. 11: Confusion matrix of the 100-epoch YOLO26n model evaluated on the cleaned single-class test set.

4.9 Confidence and Calibration Analysis

The held-out test predictions of YOLO26n-100 are summarized in Fig. 12. Most retained detections fell within the upper confidence range, with a clear concentration between 0.75 and 0.90.

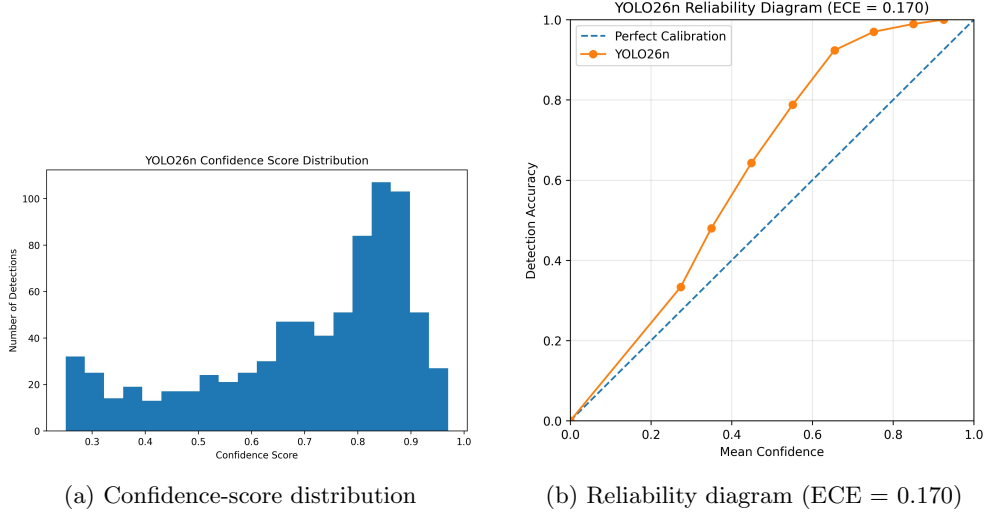


Fig. 12: Confidence and calibration analysis of the selected YOLO26n-100 model.

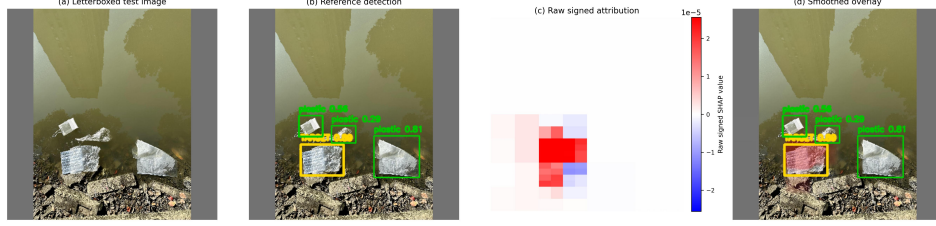
The calibration analysis used 795 detections remaining after the 0.25 confidence cutoff. These predictions were grouped into ten equal-width bins, matched to ground truth at $\text{IoU} \geq 0.50$, and summarized with ECE [35, 36].

The resulting ECE was 0.1696, with empirical precision exceeding mean confidence in all populated bins, indicating under-confidence under the adopted evaluation procedure. In the 0.80–0.90 interval, mean confidence was 0.8500 compared with an empirical precision of 0.9893. Because predictions below 0.25 and missed ground-truth objects were not included, the reported ECE represents threshold-conditioned prediction-level calibration rather than a complete localization-aware assessment.

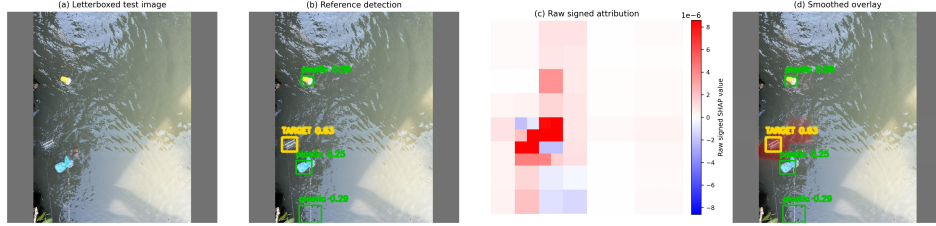
4.10 Explainable AI Analysis

SHAP-based attribution was evaluated on 21 test images using the YOLO26n-100 checkpoint [37]. For each image, the highest-confidence plastic detection above 0.25 was used as the reference target, with perturbed predictions matched to the same class at an IoU threshold of 0.30.

Fig. 13 shows two representative examples from the 21-image SHAP analysis. In both cases, attribution was distributed across the detected target and surrounding scene regions, consistent with the quantitative spatial-alignment results in Table 14.



(a) Reference detection in a cluttered near-shore scene with multiple plastic objects.



(b) Small reference target under strong water reflection and multiple-object conditions.

Fig. 13: Representative detection-specific SHAP examples generated using the YOLO26n-100 checkpoint. Each example shows the letterboxed test image, fixed reference detection, raw signed attribution map, and smoothed attribution overlay. Positive target-confidence contributions are shown in red, while negative contributions are shown in blue.

Table 14: Detection-specific SHAP faithfulness and spatial-alignment results for the selected YOLO26n-100 model ($n = 21$).

Metric	Estimate	95% CI
<i>Faithfulness</i>		
Confidence drop (20%)	0.688 ± 0.186	[0.605, 0.762]
Confidence drop (40%)	0.725 ± 0.171	[0.650, 0.797]
Confidence drop (60%)	0.729 ± 0.152	[0.663, 0.793]
<i>Spatial alignment</i>		
Target-box attribution mass	0.309 ± 0.237	[0.211, 0.411]
Expanded-box attribution mass	0.372 ± 0.262	[0.260, 0.483]
Top-5% positive pixels	0.304 ± 0.343	[0.173, 0.470]
Pointing-game hit	0.571 (12/21)	[0.365, 0.755]
Centroid hit	0.571 (12/21)	[0.365, 0.755]

Note: Continuous metrics are reported as mean $\pm$ population standard deviation with bootstrap 95% confidence intervals. Binary spatial metrics are reported as proportions with Wilson 95% confidence intervals.

Table 14 shows that masking the top 20% of positively attributed pixels reduced matched-target confidence by 0.688 ± 0.186 , with only small additional reductions at the 40% and 60% levels. Positive attribution was only partly concentrated within the target region, with $30.9\% \pm 23.7\%$ of the attribution mass inside the original box and $37.2\% \pm 26.2\%$ inside the expanded box. The pointing-game and centroid criteria were satisfied in 12 of 21 images. These results indicate that the analyzed detections relied on both target-localized and surrounding contextual information. The SHAP findings are limited to the 21 analyzed test images and to the chosen blur perturbation scheme; they should therefore be treated as sample-level attribution evidence rather than a general account of the detector’s decision process.

4.11 Qualitative Detection Analysis

Representative single-class and multiclass detections were examined to complement the quantitative evaluation.

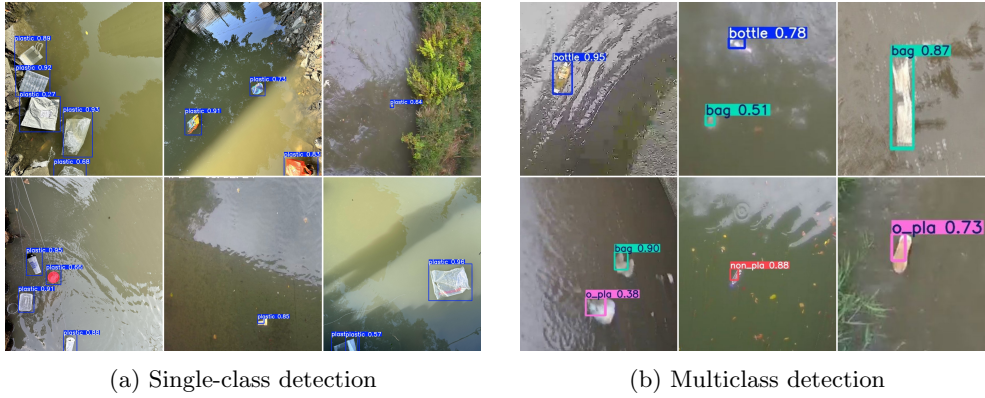


Fig. 14: Representative qualitative detection results under riverine environmental conditions.

Fig. 14 shows successful detections across variations in illumination, object scale, water texture, and background complexity, while difficult cases remained for reflective surfaces, partial submergence, and small distant objects. These selected examples illustrate model behavior and are not treated as independent evidence of robustness or generalization.

4.12 Deployment-Oriented Model Evaluation

For deployment-oriented evaluation, YOLO26n-100 was compared with YOLO26s using model complexity, checkpoint size, CPU throughput, and detection accuracy.

YOLO26n-100 used 2.38M parameters, 5.20 GFLOPs, and a 5.13 MB checkpoint, whereas YOLO26s required 9.47M parameters, 20.50 GFLOPs, and 19.36 MB (Table 15). The smaller model also recorded 0.915 mAP@0.5, 0.608 mAP@0.5:0.95,

Table 15: CPU workstation benchmark of the selected YOLO26 deployment configurations.

Model	Params (M)	GFLOPs	Size (MB)	FPS	mAP50	mAP50-95
YOLO26n-100	2.38	5.20	5.13	7.96	0.915	0.608
YOLO26s	9.47	20.50	19.36	3.30	0.876	0.580

and 7.96 FPS on the CPU benchmark. These results supported its selection as the deployment candidate. However, this comparison uses 100-epoch YOLO26n and 50-epoch YOLO26s checkpoints and should therefore be interpreted as a final-checkpoint comparison rather than an equal-training-budget experiment.

4.13 Prototype Integration and Proof-of-Concept Validation

The selected YOLO26n-100 model was integrated into the PLADICS prototype to demonstrate the detection-to-control workflow on a physical rover platform.

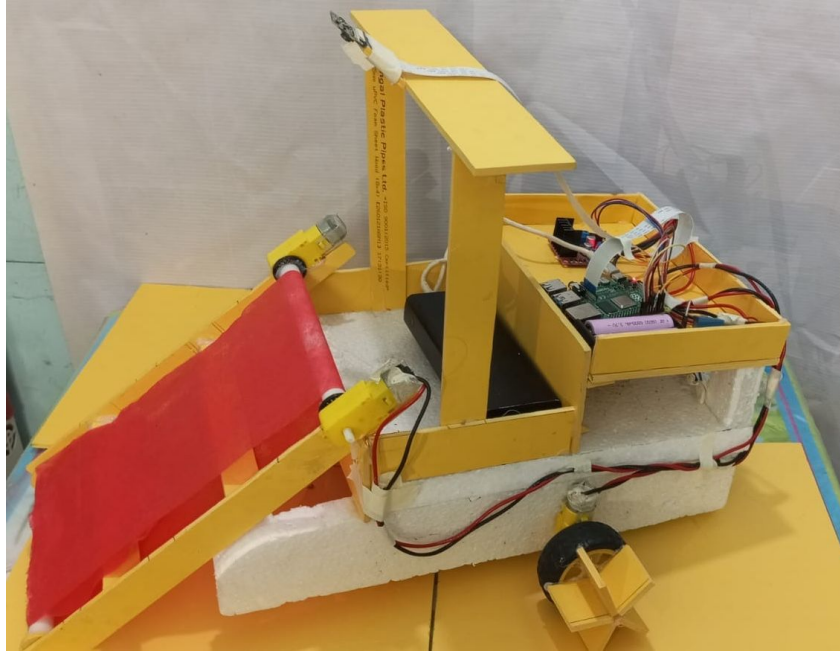


Fig. 15: Developed PLADICS prototype integrating the onboard detection, control, and plastic collection mechanisms.

The prototype demonstrates integration of the detection, control, and collection components within the PLADICS platform. The present evaluation is limited to proof-of-concept integration and does not represent long-term autonomous field validation.

5 Conclusion

This study presented PLADICS, an integrated framework for river-surface plastic detection and collection. The framework combined dataset refinement, detector comparison, confidence calibration, SHAP-based attribution, ByteTrack-assisted target continuity, rule-based rover control, NCNN inference, and IoT monitoring within a single detection-to-control workflow. The 100-epoch YOLO26n configuration emerged as the preferred lightweight model, combining low computational demand with test-set scores of 0.919 Precision, 0.844 Recall, 0.915 mAP@0.5, and 0.608 mAP@0.5:0.95. The selected model was also deployed on the Raspberry Pi-based prototype to demonstrate the integration of visual detection, navigation, and collection control. The current system still faces difficulties with small, distant, partially submerged, or occluded plastic objects, as well as strong reflections, vegetation, and visually similar background regions. Navigation is currently based on local rule-based decisions, SHAP analysis is performed offline, and monitoring is limited to a local network. Future work will investigate larger river-specific datasets, higher-resolution and multi-scale training, hard-negative sampling, domain adaptation, improved tracking, additional sensing modalities, and more advanced navigation methods. Further field testing across different rivers, seasons, cameras, and illumination conditions will be needed to assess the system under longer-term autonomous operation.

Declarations

Ethical Approval

Not Applicable.

Consent to Participate

Not Applicable.

Consent to Publish

Not Applicable.

Funding

The authors received no specific funding for this work.

Conflict of Interest/Competing Interests

The authors declare that they have no competing interests.

Data Availability

The datasets used and analyzed during the current study are publicly available and can be accessed at the following links:

- Dataset 1: River Surface Image Dataset for Instance Segmentation of Floating Macroplastic Debris. Data in Brief 63, 112189 (2025) <https://doi.org/10.1016/j.dib.2025.112189>
- Dataset 2: Riverine Plastic Litter Dataset. Mendeley Data. Dataset (2024) <https://doi.org/10.17632/7yvrhkxz22.1>

No new datasets were generated during the current study. Additional information regarding data processing and analysis is available from the corresponding author upon reasonable request.

Materials Availability

All materials required to reproduce the reported findings are available from the corresponding author upon reasonable request.

Code Availability

The code used in this study is available from the corresponding author upon reasonable request.

Author Contributions

M.E.J.A conceptualized the study, contributed to the development of the methodology, implemented the proposed system, conducted the experiments, analyzed the results, and prepared the initial manuscript draft. J.F.A contributed to data collection and preprocessing and assisted with the experimental implementation and result organization. S.S. refined the methodology, validated the results, supervised the research, and reviewed and edited the manuscript. All authors reviewed and approved the final manuscript.

References

- [1] MacLeod, M., Arp, H.P.H., Tekman, M.B., Jahnke, A.: The global threat from plastic pollution. *Science* **373**(6550), 61–65 (2021) <https://doi.org/10.1126/science.abg5433>
- [2] United Nations Environment Programme: Turning off the tap: How the world can end plastic pollution and create a circular economy. Technical report, United Nations Environment Programme, Nairobi, Kenya (2023)
- [3] Meijer, L.J.J., Emmerik, T., Ent, R., Schmidt, C., Lebreton, L.: More than 1000 rivers account for 80% of global riverine plastic emissions into the ocean. *Science Advances* **7**(18), 5803 (2021) <https://doi.org/10.1126/sciadv.aaz5803>

- [4] Emmerik, T.H.M., Kirschke, S., Schreyers, L.J., Nath, S., Schmidt, C., Wendt-Potthoff, K.: Estimating plastic pollution in rivers through harmonized monitoring strategies. *Marine Pollution Bulletin* **196**, 115503 (2023) <https://doi.org/10.1016/j.marpolbul.2023.115503>
- [5] Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., Berg, A.C.: Ssd: Single shot multibox detector. In: Leibe, B., Matas, J., Sebe, N., Welling, M. (eds.) *Computer Vision – ECCV 2016*. Lecture Notes in Computer Science, vol. 9905, pp. 21–37. Springer, Cham (2016). https://doi.org/10.1007/978-3-319-46448-0_2 . https://doi.org/10.1007/978-3-319-46448-0_2
- [6] Mittal, P.: A comprehensive survey of deep learning-based lightweight object detection models for edge devices. *Artificial Intelligence Review* **57**, 242 (2024) <https://doi.org/10.1007/s10462-024-10877-1>
- [7] Chen, L., Zhu, J.: Water surface garbage detection based on lightweight yolov5. *Scientific Reports* **14**(6133) (2024) <https://doi.org/10.1038/s41598-024-55051-3>
- [8] Fan, Z., Qin, Z., Liu, W., Chen, M., Qiu, Z.: Ss-yolov8: A lightweight algorithm for surface litter detection. *Applied Sciences* **14**(9283) (2024) <https://doi.org/10.3390/app14209283>
- [9] Zhao, Y., Lv, W., Xu, S., Wei, J., Wang, G., Dang, Q., Liu, Y., Chen, J.: DETRs beat YOLOs on real-time object detection. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 16965–16974 (2024). <https://doi.org/10.1109/CVPR52733.2024.01605>
- [10] Cao, J., *et al.*: Edge-computing-driven internet of things: A survey. *ACM Computing Surveys* **55**(8), 1–36 (2023) <https://doi.org/10.1145/3555308>
- [11] Shehzadi, T., *et al.*: Object detection with transformers: A review. *Sensors* **25**(19), 6025 (2025) <https://doi.org/10.3390/s25196025>
- [12] Cheng, Y., Zhu, J., Jiang, M., Fu, J., Pang, C., Wang, P., Sankaran, K., Onabola, O., Liu, Y., Liu, D., Bengio, Y.: FloW: A dataset and benchmark for floating waste detection in inland waters. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 10953–10962 (2021). <https://doi.org/10.1109/ICCV48922.2021.01077>
- [13] Kataoka, T., Yoshida, T., Yamamoto, N.: Risid: River surface image dataset for instance segmentation of floating macroplastic debris. *Data in Brief* **63**, 112189 (2025) <https://doi.org/10.1016/j.dib.2025.112189>
- [14] Kataoka, T., Yoshida, T., Yamamoto, N.: Instance segmentation models for detecting floating macroplastic debris from river surface images. *Frontiers in Earth Science* **12** (2024) <https://doi.org/10.3389/feart.2024.1427132>

- [15] Wang, J., Zhao, H.: Improved YOLOv8 algorithm for water surface object detection. *Sensors* **24**(15), 5059 (2024) <https://doi.org/10.3390/s24155059>
- [16] Peng, C., He, B., Xi, W., Lin, G.: Improved YOLOv7 algorithm for floating waste detection based on GFPN and long-range attention mechanism. *Wuhan University Journal of Natural Sciences* **29**(4), 338–348 (2024) <https://doi.org/10.1051/wujns/2024294338>
- [17] Nunkhaw, M., Miyamoto, H.: An image analysis of river-floating waste materials by using deep learning techniques. *Water* **16**(10), 1373 (2024) <https://doi.org/10.3390/w16101373>
- [18] Tikász, G., *et al.*: A lightweight real-time macroplastic detection framework based on yolov8 for river monitoring applications. *Frontiers in Environmental Science* (2025) <https://doi.org/10.3389/fenvs.2025.1666271>
- [19] Sun, B., Tang, H., Gao, L., Bi, K., Wen, J.: RTDETR-MARD: A multi-scale adaptive real-time framework for floating waste detection in aquatic environments. *Journal of Marine Science and Engineering* **13**(5), 996 (2025) <https://doi.org/10.3390/jmse13050996>
- [20] Badams, B., Sheikh, U.U., Syed Abu Bakar, S.A.R., Abdul Wahab, N.: FloYO-Net: Enhancing small floating waste detection in natural waters using atrous YOLOv5s. *EMITTER International Journal of Engineering Technology* **13**(2), 214–228 (2025) <https://doi.org/10.24003/emitter.v13i2.978>
- [21] Cao, Y., Luo, H., Wang, M., Wang, Y., Yan, H., *et al.*: Enhanced YOLOv8 for accurate and efficient floating object detection on water surfaces. *Scientific Reports* **16**, 2907 (2026) <https://doi.org/10.1038/s41598-025-32817-x>
- [22] Sumon, S.I., Chowdhury, M.E.H., Chowdhury, J.-U.K., Ashraf, A., Kashem, S.B.A., Majid, M.E., Nashbat, M., Khandakar, A., Hasan-Zia, M., Kunju, A.K.A.: Floating waste detection using deep learning: a comparative study of yolo, rt-detr, and faster r-cnn. *Neural Computing and Applications* **38**, 293 (2026) <https://doi.org/10.1007/s00521-026-12051-w>
- [23] Xu, X., Zhang, C., Huang, H., Kong, L.: Improving performance for multi-category anthropogenic debris detection in river environments by using a size-tailored annotation approach and data augmentation. *Scientific Reports* **16**, 18400 (2026) <https://doi.org/10.1038/s41598-026-45309-3>
- [24] Jitnupong, B., Charoensuk, J., Bundasak, S., Somritjinda, S., Silabumrungrad, N., Suraseing, J.: Detection and classification of floating waste on water surfaces using YOLO-based algorithms. *Proceedings of Engineering and Technology Innovation* **33**, 106–119 (2026) <https://doi.org/10.46604/peti.2026.16023>
- [25] Zhou, T., Liu, J., Lian, N., Chen, Y., Liu, J., Pei, Z., Li, J., Yin, Z., Zhou, J.,

- Li, Z.: ESAK-YOLO: Multi-scale attention enhanced object detection for floating waste in complex water scenes. *Scientific Reports* (2026) <https://doi.org/10.1038/s41598-026-53167-2>
- [26] Zhang, C., Yue, J., Fu, J., Wu, S.: River floating object detection with transformer model in real time. *Scientific Reports* **15**, 9026 (2025) <https://doi.org/10.1038/s41598-025-93659-1>
- [27] Zhang, C., Xu, X., Xia, Y.: Riverine Plastic Litter Dataset. Mendeley Data. Dataset (2024). <https://doi.org/10.17632/7yvrhkxz22.1>
- [28] Gong, Y., Liu, G., Xue, Y., Li, R., Meng, L.: A survey on dataset quality in machine learning. *Information and Software Technology* **162**, 107268 (2023) <https://doi.org/10.1016/j.infsof.2023.107268>
- [29] Ultralytics: Ultralytics YOLO. Software repository (2026). <https://github.com/ultralytics/ultralytics>
- [30] Wang, A., Chen, H., Liu, L., Chen, K., Lin, Z., Han, J., Ding, G.: YOLOv10: Real-time end-to-end object detection. *arXiv preprint arXiv:2405.14458* (2024) <https://doi.org/10.48550/arXiv.2405.14458> [arXiv:2405.14458](https://arxiv.org/abs/2405.14458) [cs.CV]
- [31] Jocher, G., Qiu, J., Liu, M., Lyu, S., Akyon, F.C., Kalfaoglu, M.E.: Ultralytics YOLO26: Unified real-time end-to-end vision models. *arXiv preprint arXiv:2606.03748* (2026) <https://doi.org/10.48550/arXiv.2606.03748> [arXiv:2606.03748](https://arxiv.org/abs/2606.03748) [cs.CV]
- [32] Lyu, R.: NanoDet-Plus: Super Fast and Lightweight Anchor-Free Object Detection Model. GitHub. Software implementation (2021). <https://github.com/RangiLyu/nanodet>
- [33] OASIS Open: MQTT version 5.0. Oasis standard, Organization for the Advancement of Structured Information Standards (2019). <https://docs.oasis-open.org/mqtt/mqtt/v5.0/mqtt-v5.0.html>
- [34] OpenJS Foundation and Node-RED Contributors: Node-RED: Low-Code Programming for Event-Driven Applications. OpenJS Foundation. Software project; accessed 2 August 2026 (2026). <https://nodered.org/>
- [35] Guo, C., Pleiss, G., Sun, Y., Weinberger, K.Q.: On calibration of modern neural networks. In: *Proceedings of the 34th International Conference on Machine Learning*. PMLR, vol. 70, pp. 1321–1330 (2017). <https://proceedings.mlr.press/v70/guo17a.html>
- [36] Küppers, F., Kronenberger, J., Shantia, A., Haselhoff, A.: Multivariate confidence calibration for object detection. In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pp. 326–327 (2020).

<https://doi.org/10.1109/CVPRW50498.2020.00171>

- [37] Lundberg, S.M., Lee, S.-I.: A unified approach to interpreting model predictions. In: Advances in Neural Information Processing Systems (NeurIPS), vol. 30, pp. 4765–4774 (2017)